马晨熙

大语言模型推理系统 / 机器学习系统工程师

[北京] | [+86 15387087330] | shannon20051127@gmail.com | github.com/algebra-MCX

教育背景

武汉理工大学 06/2028

本科·机械工程

开源工程经历

vLLM Project 2026 – 至今

开源贡献者：*Model Runner V2 / DBO / MoE* GitHub

一、MRV2 DBO 执行正确性与异常处理

- **专家路由索引修复（已合并）：** #63372（Merged）。修复 MRV2 DBO 在 Microbatch 执行过程中 Routed Expert 数据写入的索引错误，使用 Microbatch 对应的 Token Offset 正确定位专家路由结果。在 32 请求的 DeepSeek-V2-Lite 测试中，路由一致率由主分支的 0% 提升至 89.4%。
- **Microbatch 异常传播（已合并）：** #63476（Merged）。改进 `UBatchRunner.finish()` 的异常处理逻辑，确保系统报告最先记录的 Microbatch 异常，避免原始错误被后续异常覆盖。新增 H200 GPU 回归测试，2/2 测试通过。
- **DBO Worker 故障恢复（Draft）：** #64718（Draft）。针对某个 Microbatch 执行失败后，其他 Worker 仍阻塞于同步等待的问题，设计异常发布与等待者唤醒机制，避免 `UBatchRunner.finish()` 永久挂起，并保留原始异常。这个因为目前 vLLM DBO 要摆脱 MRV1 的依赖，所以这个 PR 得等到我提出的 64668 合并，才会合并

二、CUDA Graph 与推理性能优化

- **MLA Decode Staging（Open）：** #63556（Open）。扩展 MRV2 的 MLA Decode Staging，支持不均衡 Data Parallel 负载下捕获的 FULL Microbatch CUDA Graph，优化 DBO 场景中的 Graph Replay 执行。已经受到 maintainer 高度重视，他提出了另一套方案，可能不会采用我的方案。
- **性能验证：**在记录的 63/97 负载 A/B 实验中，输出吞吐量提升 15.52%，p95 Time Per Output Token（TPOT）从 132.017 ms 降至 113.930 ms，约降低 13.7%。使用 GPU 基准测试比较不同执行策略的吞吐量与尾延迟表现。

三、MoE 通信正确性：NIXL / DeepEP

- **NIXL Expert Mapping（Open）：** #64086（Open）。修复 Global Expert ID 到 Physical Expert ID 映射过程中无效专家 ID 的处理问题，保留 `-1` Padding Sentinel，避免无效专家被错误转换为合法专家索引。目前已经被 Redhat 顶级工程师赞同修改，不过还没有给 ready 标签。
- **DeepEP LL Expert Mapping（Open）：** #64340（Open）。修复 DeepEP Low-Latency Dispatch / Combine 路径中 Round-Robin Expert Remapping 的 Sentinel 传播问题，统一相关映射逻辑及回归测试。目前已经被 Redhat 顶级工程师赞同修改，不过还没有给 ready 标签。
- **正确性验证：**覆盖混合有效/无效专家 ID、全 Padding 及线性映射场景；NIXL 与 DeepEP LL 两种后端共 6 项 CPU Adapter 回归测试全部通过。

四、MRV2 DBO 架构重构

- **移除 MRV1 依赖（Draft）：** #64668（Draft）。重构 MRV2 DBO 的 Microbatch Slicing 逻辑，在 MRV2 中直接计算 Padded Slices，移除对 MRV1 `maybe_create_ubatch_slices` 的调用。
- 调整 MRV2 Microbatch Slicing 测试，为 5 种 Batch Layout 固定预期结果，减少测试对 MRV1 实现的依赖，推进 MRV2 DBO 执行路径的模块解耦。

研究经历

华盛顿大学系统实验室（UW Systems Lab） 05 2026 – 至今

研究实习生（*Research Intern*） 美国西雅图 / 远程

- 作为第三作者参与 *TraceLab: Characterizing Coding Agent Workloads for LLM Serving*（arXiv:2606.30560），研究 Coding Agent 工作负载的执行模式及其对大语言模型推理服务系统的影响。
- 参与分析约 4,300 个 Coding Agent 会话、350,000 次 LLM 推理步骤和 430,000 次工具调用，研究 Agent 工作负载的请求分布、上下文长度及工具交互行为。
- 分析长时间自主执行循环、长输入与短输出、工具调用重尾分布及 Prefix Cache 命中率等特征，为 Agent 场景下的 LLM Serving 性能建模与系统优化提供实证依据。

技术技能

推理系统： vLLM、LLM Serving、MRV2、Dual-Batch Overlap、Microbatch Execution、CUDA Graphs、Prefill / Decode

分布式与 MoE： Data Parallelism (DP)、Expert Parallelism (EP)、NIXL、DeepEP、Expert Routing、Dispatch / Combine

性能与正确性： GPU Profiling、Benchmarking、Throughput / TPOT、CUDA Graph Replay、Regression Testing、Failure Propagation

开发工具： Python、PyTorch、pytest、Git / GitHub、Linux、NVIDIA H200 / B200